The following listing provides a closer look at its code that represents this functionality (use-chat-form-submit.js).

Listing 2.3 Function that submits the current message

```
import { useState } from "react";
import { generateUniqueId } from "@/lib/generateUniqueId";

function useChatFormSubmit(getAssistantResponse) {
  const [messages, setMessages] = useState([]);
  const [inputValue, setInputValue] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const value = inputValue.trim();
    if (!value) return;

    setIsLoading(true);
    setInputValue("");

    const userMessage = {
      content: value,
      role: "user",
      id: generateUniqueId(),
    };
    setMessages((currentMessages) =>
[...currentMessages, userMessage]);
    try {
      const {message} = await getAssistantResponse(value);
        setMessages((currentMessages) =>
[...currentMessages, responseMessage]);
    } catch (error) {
      console.error(error);
    } finally {
      setIsLoading(false);
    }
  };

  return { messages, isLoading, handleSubmit, inputValue, setInputValue };
}

export default useChatFormSubmit;
```

Messages state: holds the conversation history

Input state: current user text

Loading state: indicates assistant is processing

Trim input: removes leading/trailing spaces

Prevents empty messages

Sets loading before fetching response

Clears input after sending

Adds user message to state

Fetches assistant response

Adds assistant response to state

Logs errors

Resets loading after processing

This hook manages the chat state, including all messages, the current input, and a loading indicator. `handleSubmit` processes user input by preventing empty messages, adding the user's message to the state, fetching the assistant's response, and updating the messages again. Errors are logged, and the loading state is reset once processing completes. By encapsulating this logic, the hook keeps the component clean and reusable, making it easy to handle chat interactions consistently across the app.